Fakultas Informatika
School of Computing
Telkom University

Aplikasi Berbasis Platform

Back-end & MVC

Server Side Scripting (Web Back-end)

- Massive amounts of information makes it impractical to custom-build every single page using only HTML, CSS, and Javascript.
- Use programming language (server side scripting) for data and logical processing in backend
- Most backends using database to manage the flow of information sent to the user

Programming Language for Web

- PHP
- Java
- NodeJS
- C# / Basic (ASP .NET)
- Ruby
- Python
- Go

DBMS Type

- Relational
  - Organize data into tables, where data is structured in rows and columns
- Non-Relational
  - Stores data using document, graph, or key-value formats

DBMS Examples

PostgreSQL

MySQL®

cassandra

ORACLE®
DATABASE

mongoDB®

Need more corporate and power? You can try SawitDB Powered by WowoEngine

Notes in Server Side Scripting: Database

- How to access your data in database
  - Connection: local server (LAN) or remote server (GCP, AWS)
  - Query: SQL native or using Object Relational Model (Hibernate, EJB)
- DBMS uses SQL and other SQL inheritance
  - Some DBMS has their own additional self-made query
- Most common is using Relational DB
  - MySQL / MariaDB, PostgreSQL, Oracle, SQL Server
- Using Non-Relational DB is good too, but will make the implementation of DB schema more complex
  - MongoDB, Google Firebase / Firestore

Notes in Server Side Scripting: Designing URLs for Readability

Using **Nouns** to Represent Resources

- /getProducts ❌
- /products ✅

Avoid adding **file extensions** in URLs

- products.json ❌
- products ✅

Using **Plural** for Resource Collection

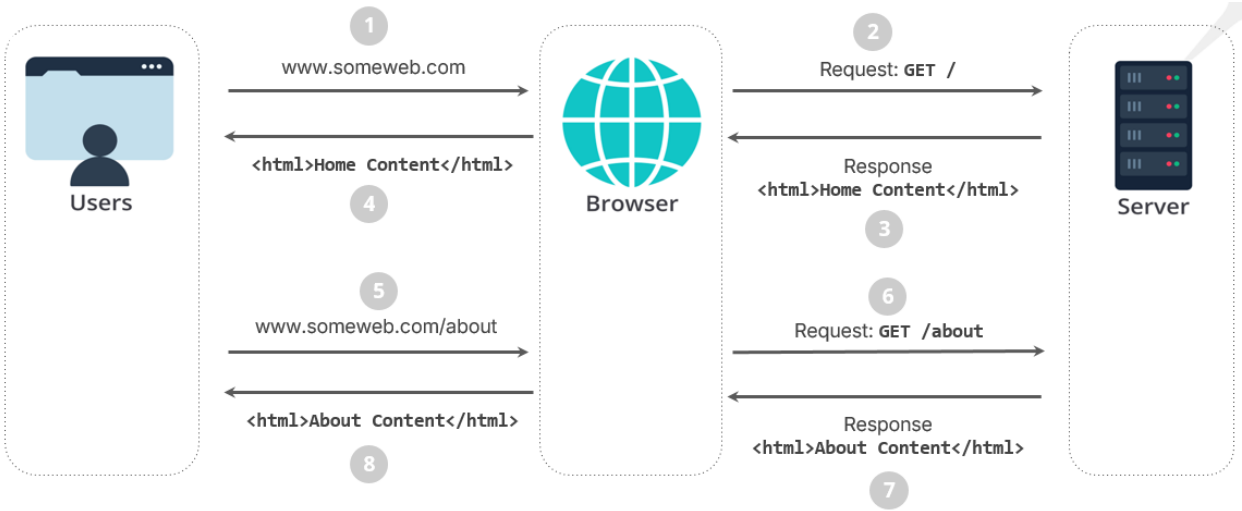
- /product ❌
- /products ✅

Using **Hyphens** to Separate Words

- /checkoutorder ❌
- /checkout-order ✅

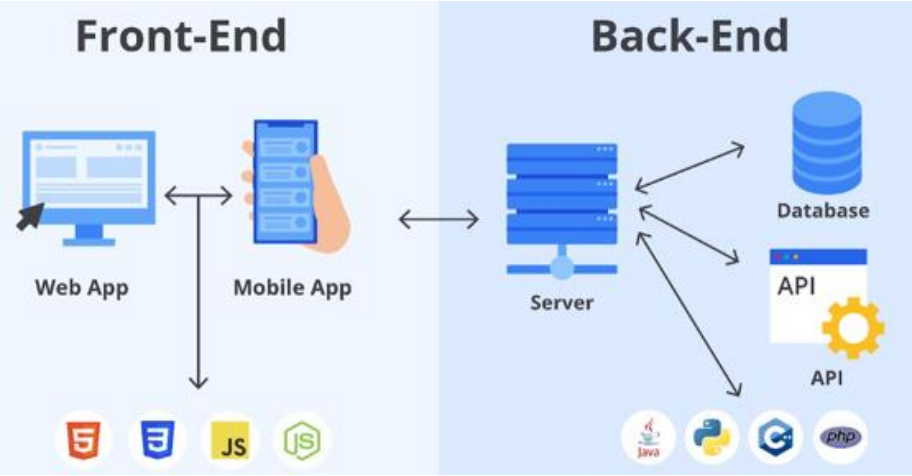
How to build Front-end and Back-end

- Server-Side Rendering
  - Backend loads a frontend file, fills it with data from backend (optional, if any), generates the file into a HTML page, and returns the page to browser
  - Usually used in monolithic development, where backend and frontend codes are in the same repository/codebase



How to build Front-end and Back-end

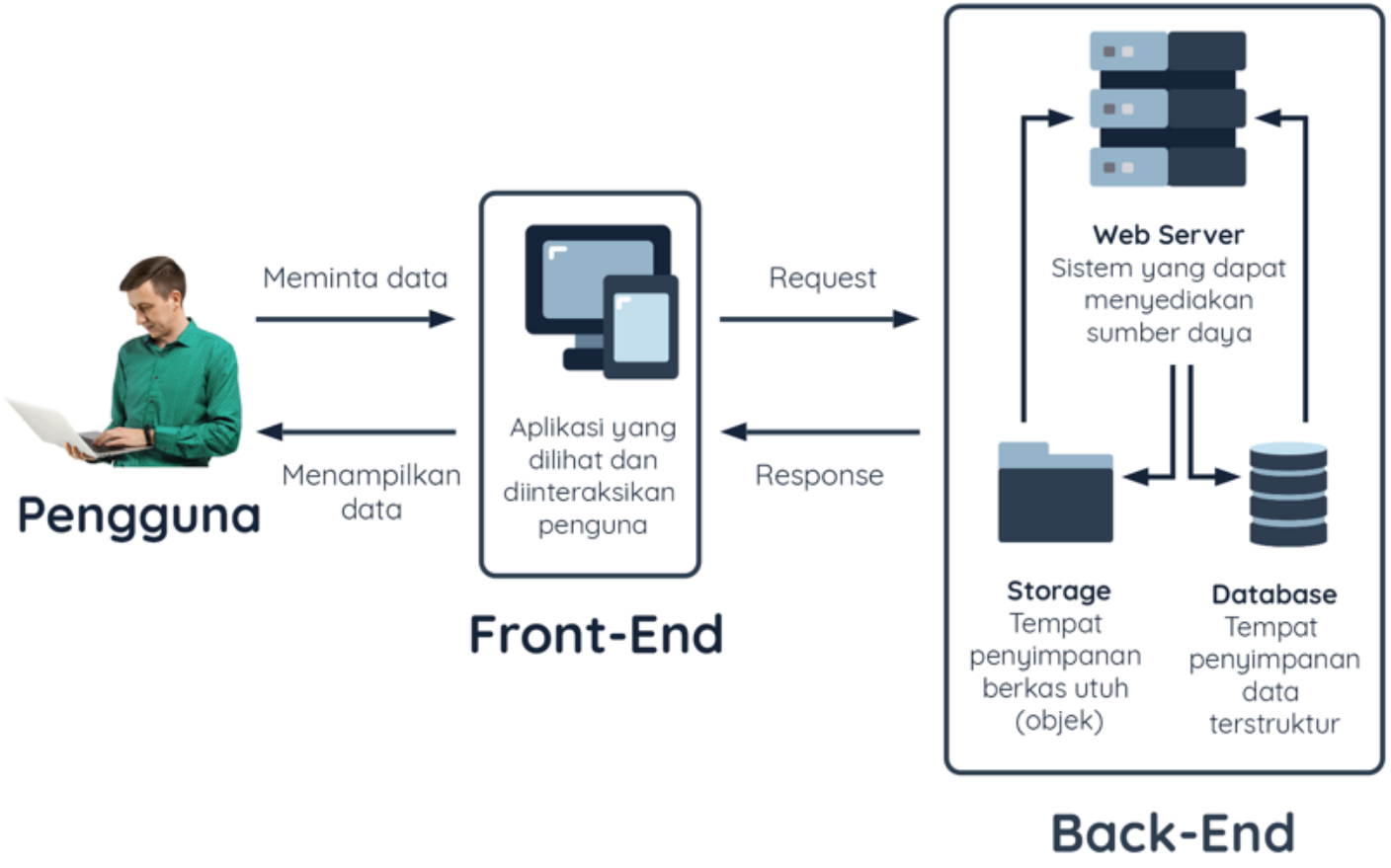
- Client-Side Rendering
  - Backend doesn't return a page, instead returns a formatted data to browser
  - Usually used in API-driven or microservices development (frontend and backend codes are in separate/different repository/codebase



How web applications "talk"?

- They communicate using HyperText Transfer Protocol (HTTP).
- It is a request-response pattern, meaning a request must be made to receive a response.

In Client-Side Rendering or API-driven, how back-end and front-end "talk"?



HTTP Language: methods

- HTTP methods

Method	Description
GET	retrieving information.
POST	adding data.
PUT	modifying data.
DELETE	deleting data.

HTTP Language: status code

- HTTP status code

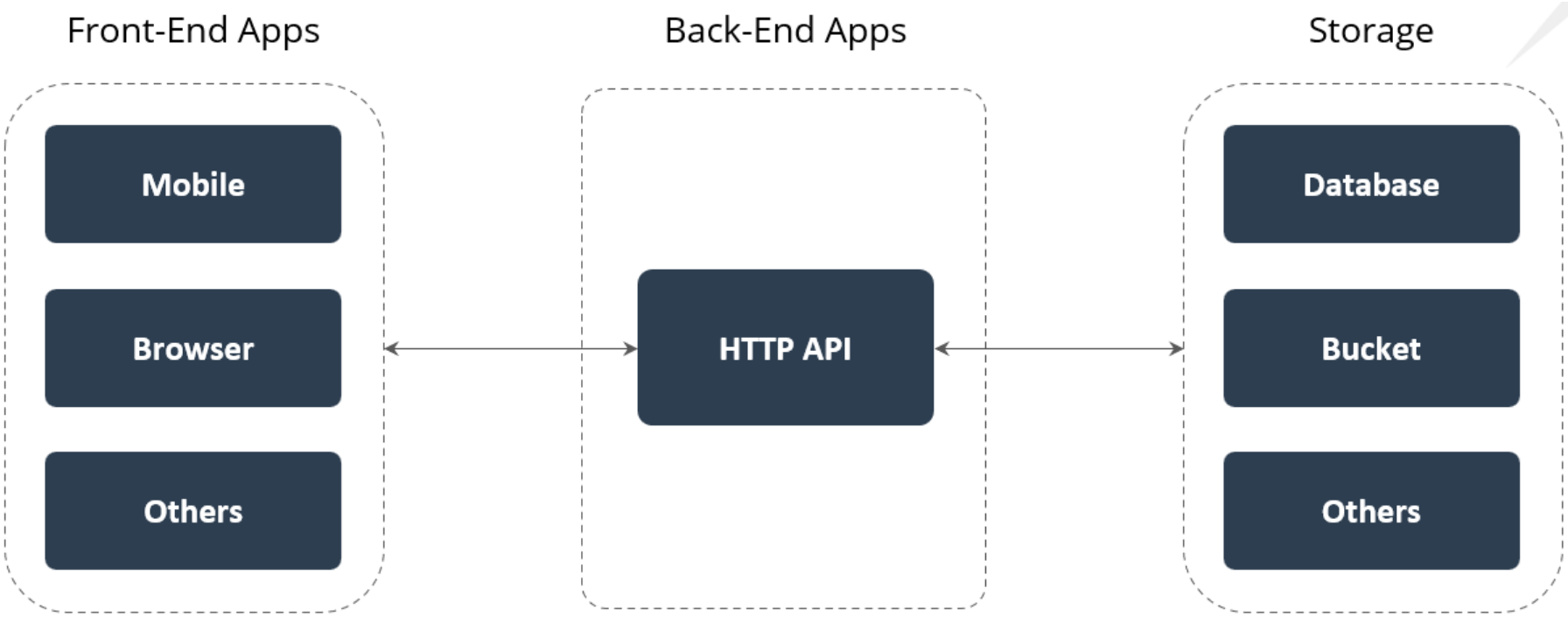
Status Code	Description
100 – 199	Informational responses
200 – 299	Successful responses
300 – 399	Redirection messages
400 – 499	Client error responses
500 – 599	Server error responses

- Example:
  - 200 (OK)
  - 201 (Created)
  - 400 (Bad Request)
  - 401 (Unauthorized)
  - 403 (Forbidden)
  - 404 (Not Found)
  - 500 (Internal Server Error)

HTTP API

- Application Programming Interface (API) is an interface that help an app communicate with another apps.
- HTTP API bridges communication between web applications: frontend-backend or backend-backend
- Not only web, they can be part of SDKs (Software Development Kits) and other forms of interfaces.

HTTP API


Front-End Apps
Back-End Apps
Storage
Mobile
Browser
Others
HTTP API
Database
Bucket
Others

Model-View-Controller (MVC)

- MVC is an architecture pattern for software that separates the application into three roles:
  - Model (business logic)
  - View (user interface)
  - Controller (process logic / user input)

- This separation of roles allows software development, testing, and maintenance for each role to be more flexible and independent

Model

- Manage the data behavior of application scope
- Responds request from Controller to do data management process
- Example: class that represents database table, active record

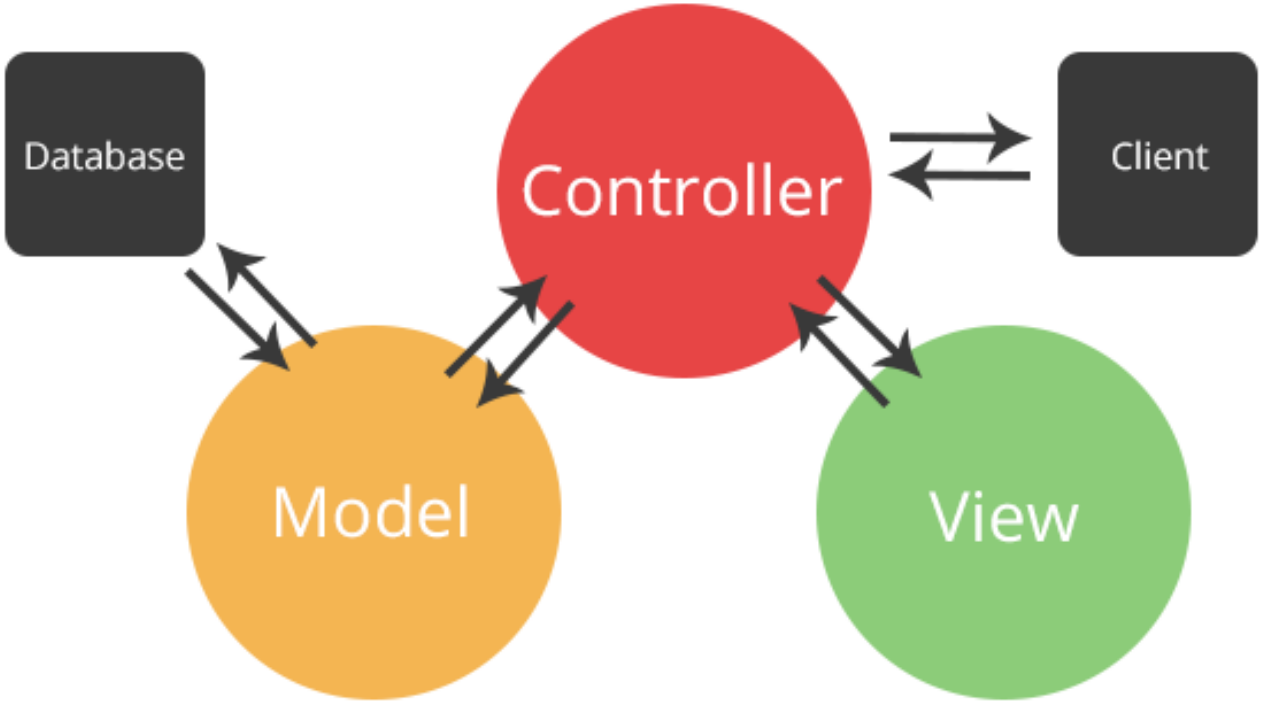
View

- The UI or forms to interact with user
- One Controller can use many Views for different needs
- Example: login page, registration form, dashboard page

Controller

- Accepts and process the user inputs from View
- Responds with Model or View calling
- Give instructions to Model / View to do some action
- Example: Class that isn't a Model or a View, usually contains logic and business process

MVC

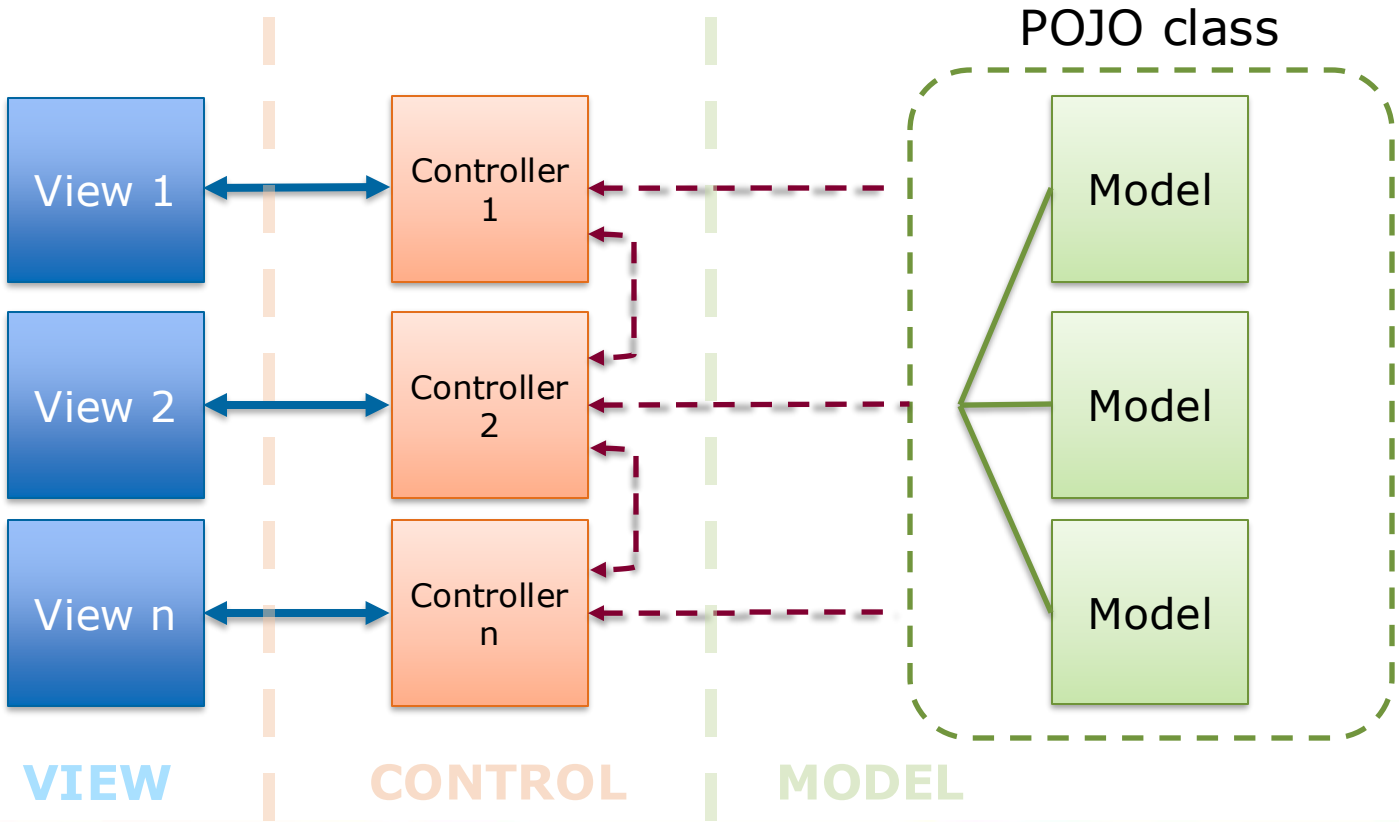


MVC Flow

- User opens a URL address, Controller that mapped to that address returns the View
- User interacts with View (UI) (example: fill the input field or click a button)
- Controller handles the inputs from View, runs the logic / calls and instructs the Model
- Model runs the instructions
- Model give a return message / data to Controller
- Controller return the message / data to View and/or return another View

MVC Approach : n view - n controller

Most projects and frameworks now



Any question?

Fakultas Informatika
School of Computing
Telkom University

THANK YOU